

Implementing Complex YOLOv4-PyTorch — An End-to-End Journey

Open in app ↗

Sign up

Sign in

Medium

Search

Write



...



Overview

In this article, I'll walk you through how I implemented the Complex-YOLOv4-PyTorch object detection model. The goal of this project is to detect and localize 3D objects such as **Car, Van, Pedestrian, and Cyclist** from the popular KITTI dataset.

Complex-YOLOv4 is an enhanced version of YOLOv4 that supports **3D object detection in Bird's Eye View (BEV)** by integrating spatial and orientation-aware features. It's a powerful deep learning model that can be deployed in autonomous driving systems for real-time perception.

Contents:

1. System Requirements
2. Required Dependencies
3. Dataset Preparation
4. Training and Testing
5. Results
6. Credits

. . .

1. System Requirements

1. **Operating System:** Windows 10 or Linux
2. **GPU:** NVIDIA GPU with at least **4 GB VRAM** (CUDA enabled)
3. **CUDA Toolkit:** Version **10.2** or **11.x**
4. **Python Version:** **3.6** to **3.9**
5. **PyTorch Version:** **1.7** to **1.10** (compatible with selected CUDA version)
6. **Anaconda Prompt**

7. **RAM:** At least 8 GB

8. **Disk Space:** Minimum 30 GB (for KITTI dataset, logs, and results)

. . .

2. Required Dependencies

Clone the github repository from [here](#). Open Anaconda Prompt and navigate to the folder of the cloned repository. Next, create a conda environment (yolo_legacy) and install the required dependencies through `pip install <dependency name> .`

```
# Create a new Conda environment
```

```
conda create -n yolo_legacy python=3.7
```

```
conda activate yolo_legacy
```

```
# Essential dependencies
```

```
pip install torch==1.5.0+cu101 torchvision==0.6.0+cu101 -f https://download.pyto
```

```
pip install numpy opencv-python==4.2.0.32
```

```
pip install easydict
```

```
pip install tqdm
```

```
pip install tensorboard
```

```
pip install six
```

```
pip install shapely
```

```
pip install scipy
```

```
pip install matplotlib
```

```
pip install mayavi
```

. . .

3. Dataset Preparation

Download the official KITTI Dataset from [here](#). The required files to be downloaded are:

- Velodyne point clouds (29 GB)

- Training labels of object data set (5 MB)
- Camera calibration matrices of object data set (16 MB)
- Left color images of object data set (12 GB)

Each dataset file consists of `train` and `test` subfolders. Once the files mentioned above are downloaded, navigate to the `dataset` subfolder of the cloned repository and paste the files.

```
Complex-YOLOv4-PyTorch/
├── config/
├── data_process/
├── dataset/
│   └── kitti/          <-- Paste image, calib, velodyne, label files here
├── models/
├── checkpoints/
├── results/
├── src/
│   ├── train.py
│   └── test.py
└── README.md
```

. . .

4. Training and Testing

4.1 Training

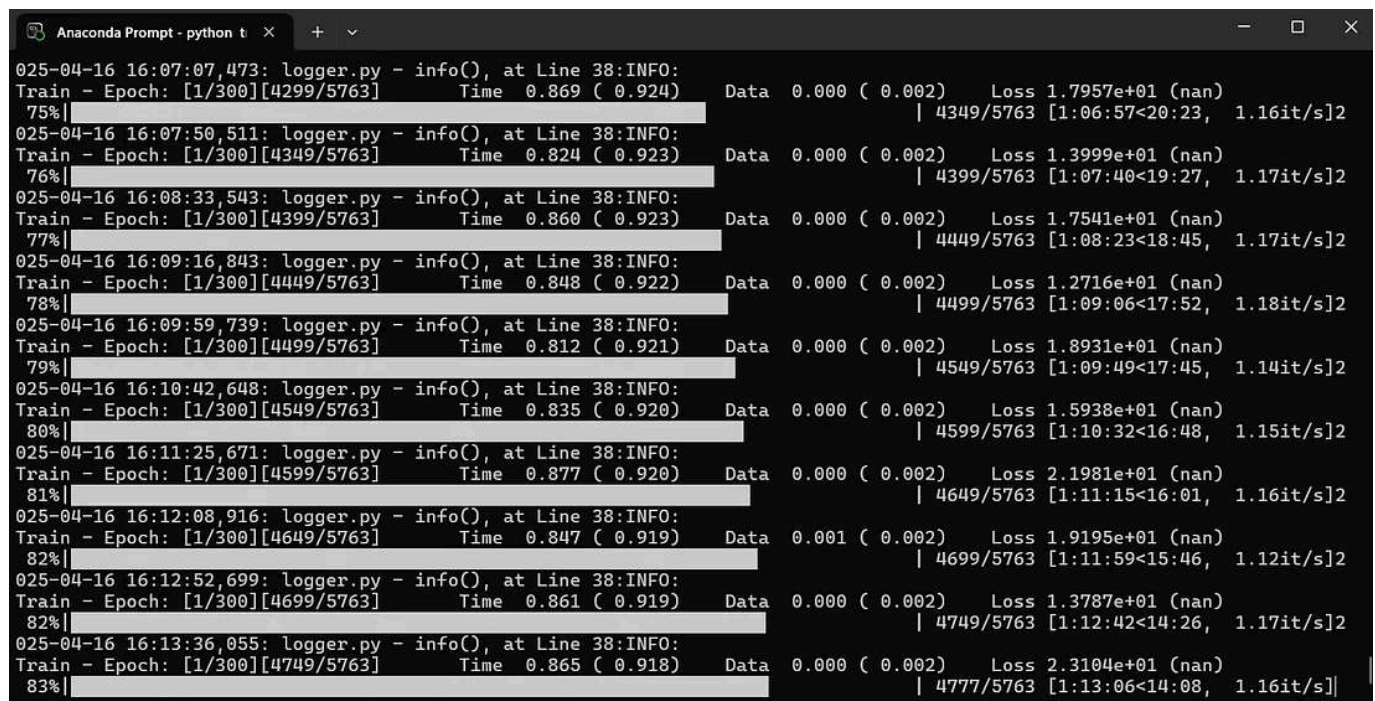
The `train` and `test` files are located in the `src` folder of the repository. In the Anaconda Prompt navigate to the `src` folder and execute the following command.

```
python train.py --cfgfile <path_to_config> --gpu_idx 0 --batch_size 8 --num_work
```

To execute the `train.py` script you will need a `.cfg` file which can be found in the path below,

```
Complex-YOLOv4 Pytorch\src\config\cfg\complex_yolov4.cfg
```

The `--batch_size` and `- num_workers` can be varied with respect to the computational capacity of your system. The model is trained for 300 epochs by default. However, `- num_epochs` can be set to the required number of epochs under 300. Alternatively, you can also use `ctrl+c` for early termination of the training process.



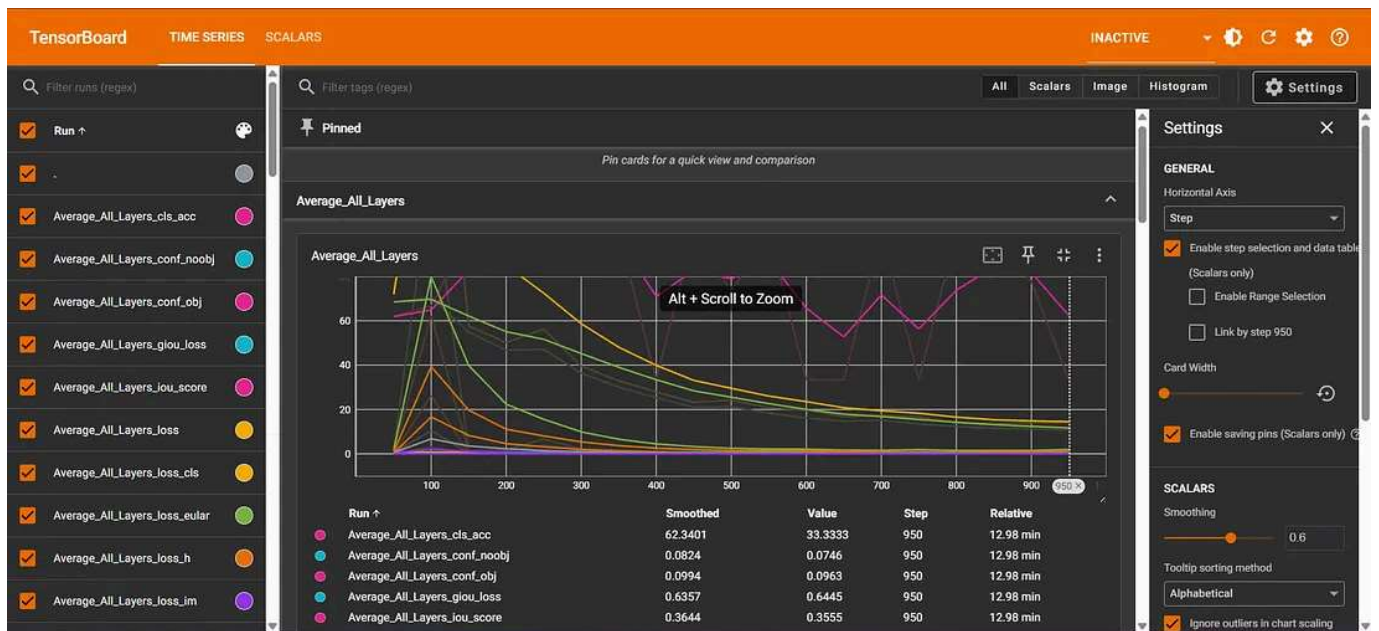
```
025-04-16 16:07:07,473: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4299/5763] Time 0.869 (0.924) Data 0.000 (0.002) Loss 1.7957e+01 (nan)
75%| 4349/5763 [1:06:57<20:23, 1.16it/s]2
025-04-16 16:07:50,511: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4349/5763] Time 0.824 (0.923) Data 0.000 (0.002) Loss 1.3999e+01 (nan)
76%| 4399/5763 [1:07:40<19:27, 1.17it/s]2
025-04-16 16:08:33,543: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4399/5763] Time 0.860 (0.923) Data 0.000 (0.002) Loss 1.7541e+01 (nan)
77%| 4449/5763 [1:08:23<18:45, 1.17it/s]2
025-04-16 16:09:16,843: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4449/5763] Time 0.848 (0.922) Data 0.000 (0.002) Loss 1.2716e+01 (nan)
78%| 4499/5763 [1:09:06<17:52, 1.18it/s]2
025-04-16 16:09:59,739: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4499/5763] Time 0.812 (0.921) Data 0.000 (0.002) Loss 1.8931e+01 (nan)
79%| 4549/5763 [1:09:49<17:45, 1.14it/s]2
025-04-16 16:10:42,648: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4549/5763] Time 0.835 (0.920) Data 0.000 (0.002) Loss 1.5938e+01 (nan)
80%| 4599/5763 [1:10:32<16:48, 1.15it/s]2
025-04-16 16:11:25,671: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4599/5763] Time 0.877 (0.920) Data 0.000 (0.002) Loss 2.1981e+01 (nan)
81%| 4649/5763 [1:11:15<16:01, 1.16it/s]2
025-04-16 16:12:08,916: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4649/5763] Time 0.847 (0.919) Data 0.001 (0.002) Loss 1.9195e+01 (nan)
82%| 4699/5763 [1:11:59<15:46, 1.12it/s]2
025-04-16 16:12:52,699: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4699/5763] Time 0.861 (0.919) Data 0.000 (0.002) Loss 1.3787e+01 (nan)
82%| 4749/5763 [1:12:42<14:26, 1.17it/s]2
025-04-16 16:13:36,055: logger.py - info(), at Line 38:INFO:
Train - Epoch: [1/300][4749/5763] Time 0.865 (0.918) Data 0.000 (0.002) Loss 2.3104e+01 (nan)
83%| 4777/5763 [1:13:06<14:08, 1.16it/s]
```

Terminal Window of Training Process

The model state dictionaries are saved after every 5 epochs in the form of a `.pth` file and can be found in the directory, `Complex-YOLOv4`

```
Pytorch\checkpoints\complexer_yolo\Model_complexer_yolo_epoch_5.pth
```

This can be used to load the model states from the `.pth` file directly for inference.



Tensorboard Dashboard

During training, to monitor the performance of the training process across various metrics open a new Anaconda Prompt terminal and execute the code below.

```
cd Complex-YOLOv4-Pytorch\logs\complexer_yolo\tensorboard
tensorboard --logdir=.
```

Next, go to <http://localhost:6006/> this will open the Tensorboard window where real-time performance of the training process can be observed across various performance metrics such as accuracy, IoU, loss, precision etc. The corresponding values can also be downloaded in the form of a .csv or JSON file.

4.2 TESTING

Once, the training operation is complete, we will now test the trained model. To test the model on the evaluation set we need to run the code below,

```
python test.py --pretrained_path <path_to_model> --save_test_output --gpu_id 0
```

Ensure to paste the path of the `.pth` file which has the saved model states obtained during the training process. You can directly test for inference by downloading a pretrained model from the repository.

```
Done testing the 7488th sample, time: 163.7ms, speed 6.11FPS
Done testing the 7489th sample, time: 157.0ms, speed 6.37FPS
Done testing the 7490th sample, time: 144.1ms, speed 6.94FPS
Done testing the 7491th sample, time: 141.4ms, speed 7.07FPS
Done testing the 7492th sample, time: 141.0ms, speed 7.09FPS
Done testing the 7493th sample, time: 161.4ms, speed 6.19FPS
Done testing the 7494th sample, time: 157.0ms, speed 6.37FPS
Done testing the 7495th sample, time: 152.0ms, speed 6.58FPS
Done testing the 7496th sample, time: 157.0ms, speed 6.37FPS
Done testing the 7497th sample, time: 155.8ms, speed 6.42FPS
Done testing the 7498th sample, time: 141.4ms, speed 7.07FPS
Done testing the 7499th sample, time: 156.6ms, speed 6.38FPS
Done testing the 7500th sample, time: 157.0ms, speed 6.37FPS
Done testing the 7501th sample, time: 141.1ms, speed 7.09FPS
Done testing the 7502th sample, time: 141.4ms, speed 7.07FPS
Done testing the 7503th sample, time: 156.7ms, speed 6.38FPS
Done testing the 7504th sample, time: 156.6ms, speed 6.38FPS
Done testing the 7505th sample, time: 157.1ms, speed 6.37FPS
Done testing the 7506th sample, time: 141.2ms, speed 7.08FPS
Done testing the 7507th sample, time: 156.7ms, speed 6.38FPS
Done testing the 7508th sample, time: 141.0ms, speed 7.09FPS
Done testing the 7509th sample, time: 156.7ms, speed 6.38FPS
Done testing the 7510th sample, time: 141.0ms, speed 7.09FPS
Done testing the 7511th sample, time: 157.0ms, speed 6.37FPS
Done testing the 7512th sample, time: 157.0ms, speed 6.37FPS
Done testing the 7513th sample, time: 157.1ms, speed 6.37FPS
Done testing the 7514th sample, time: 156.7ms, speed 6.38FPS
Done testing the 7515th sample, time: 156.7ms, speed 6.38FPS
Done testing the 7516th sample, time: 149.5ms, speed 6.69FPS
Done testing the 7517th sample, time: 157.0ms, speed 6.37FPS
(yolo_legacy) C:\Users\mbavi\Downloads\Complex-YOLOv4-Pytorch\src>
```

Terminal Window of Testing Process

You can also toggle:

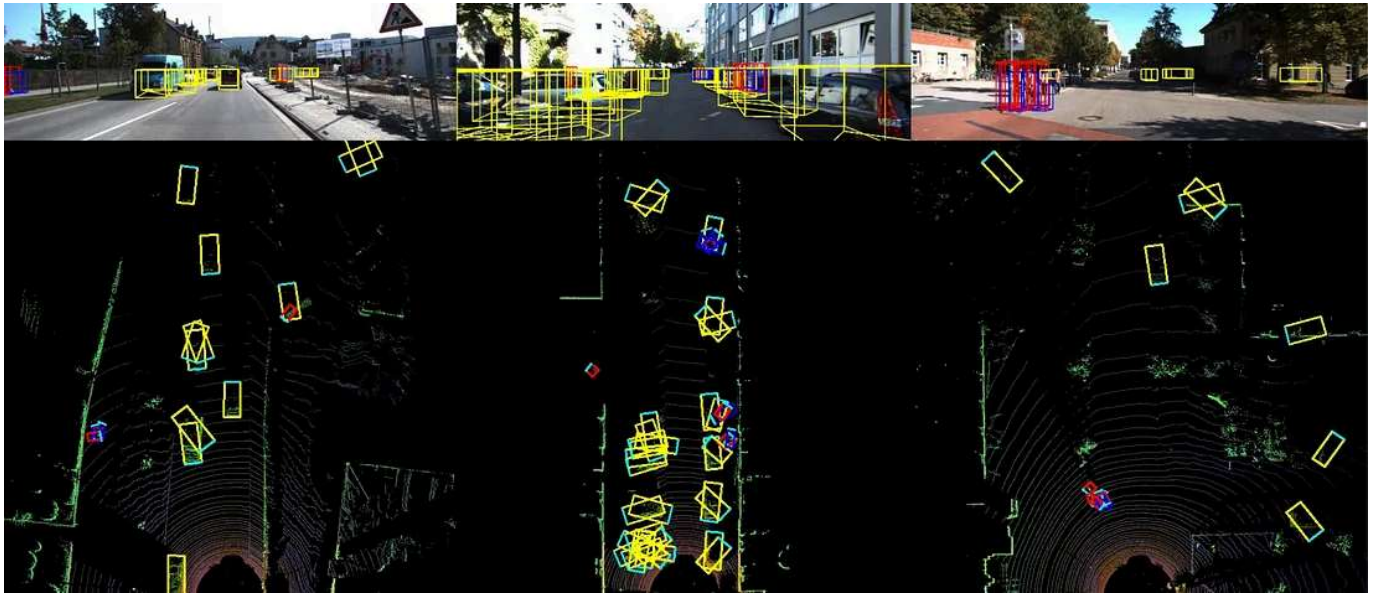
- `--show_image` for visual display
- `--output_format video` for video output

. . .

5. Results

Once you test your model using the provided test script, you can find the output results in the path `Complex-YOLOv4-Pytorch\results`

This folder contains image files with detection results for each test sample from the KITTI dataset.



RGB + BEV Visualisation

Each output image includes:

- **Bird's Eye View (BEV)** of the LiDAR point cloud, overlaid with rotated bounding boxes that indicate object positions and orientations.
- **RGB camera image** with 3D bounding boxes projected onto it for visual alignment with real-world perspective.

. . .

6. Credits

This implementation is based on the open-source project by **Nguyen Mau Dung**, hosted on GitHub: [Complex-YOLOv4-PyTorch Repository](#).

✉ *Author: Nguyen Mau Dung*

💬 *Email: nguyenmaudung93.kstn@gmail.com*

Full credit goes to the original author and contributors of the Complex-YOLOv4 repository. This article is designed to serve as a clear, beginner-friendly guide that simplifies the implementation process, helping users understand, set up, and run the model with ease.

- 3d Object Detection
- Yolov4
- Kitti Dataset
- Pytorch
- Lidar



Written by Bavitesh M
0 Followers · 0 Following

Follow

No responses yet 



Write a response

What are your thoughts?

Recommended from Medium

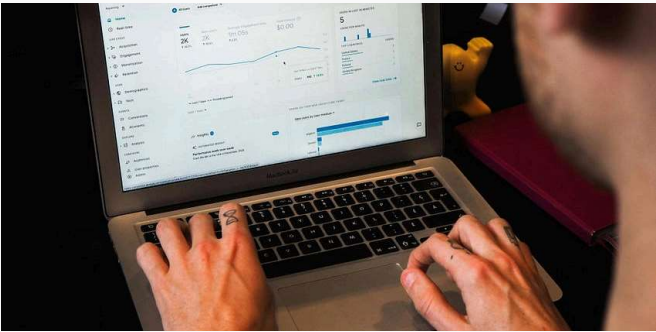


 In Level Up Coding by Sahib Dhanjal

PyTorch—A Comprehensive Performance Tuning Guide

Best practices used to develop fast and clean scalable code

★ Apr 17 🖱️ 302

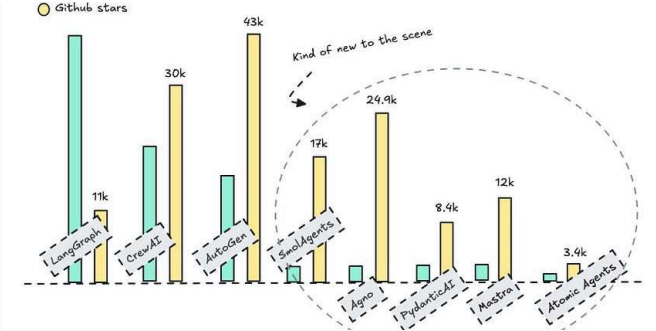
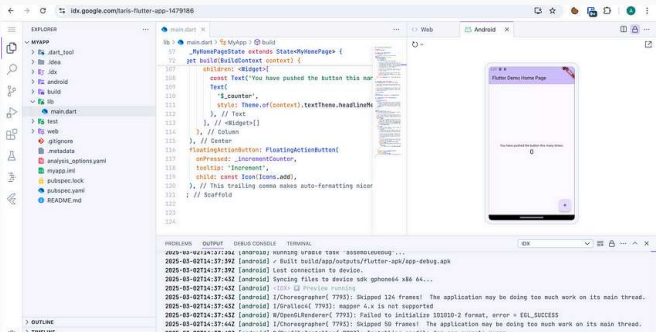


 pritesh

Data Analysis part-intro

hello,

★ 3d ago 🖱️ 1

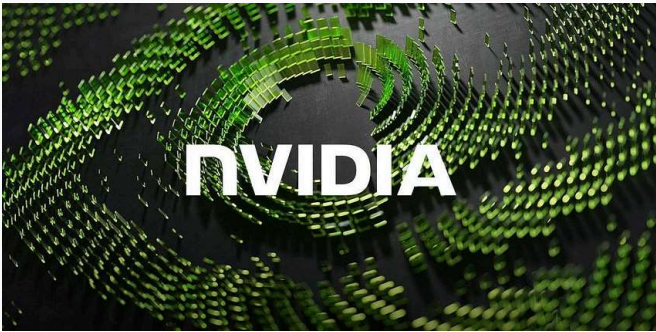


 In Data Science Collective by Ida Silfverskiöld

Agentic AI: Comparing New Open-Source Frameworks

By looking at functionality and learning curves

★ Apr 15 🖱️ 1K 💬 33



 In Python in Plain English by Dylan Cooper

A New Era for GPU Programming: NVIDIA Finally Adds Native Pytho...

This year, NVIDIA is going all in, making it clear that Python is to become a first-class...

★ Apr 17 🖱️ 405 💬 3





In Coding Beauty by Tari Ibaba



Nandini Lokesh Reddy

This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.



Mar 12



4.8K



272



Apr 4



21



YOLO-E: In Detail

Computer vision has made tremendous strides in recent years, with models like YOL...

See more recommendations